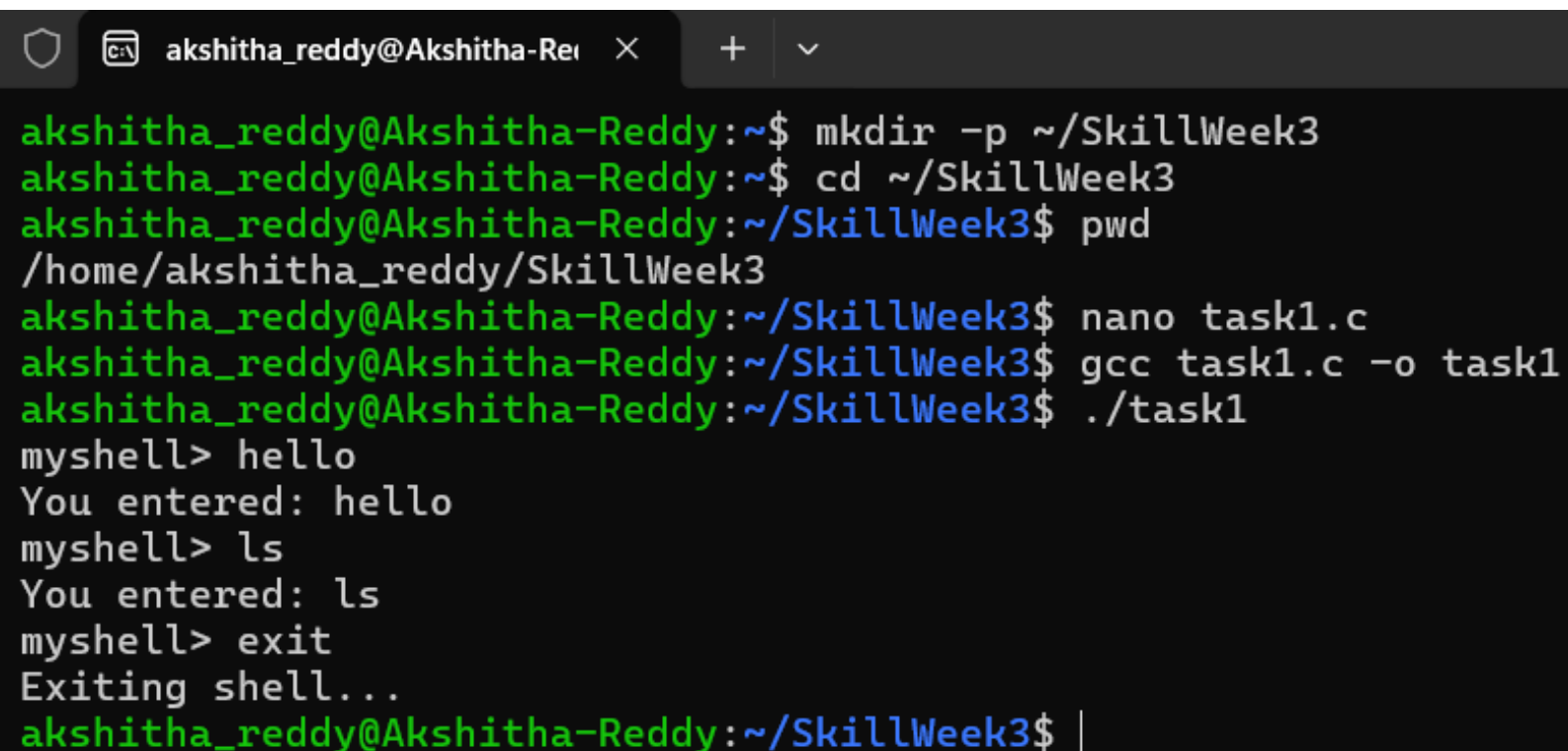




```
akshitha_reddy@Akshitha-Reddy:~$ mkdir -p ~/SkillWeek3
akshitha_reddy@Akshitha-Reddy:~$ cd ~/SkillWeek3
akshitha_reddy@Akshitha-Reddy:~/SkillWeek3$ pwd
/home/akshitha_reddy/SkillWeek3
akshitha_reddy@Akshitha-Reddy:~/SkillWeek3$ nano task1.c
akshitha_reddy@Akshitha-Reddy:~/SkillWeek3$ gcc task1.c -o task1
akshitha_reddy@Akshitha-Reddy:~/SkillWeek3$ ./task1
myshell> hello
You entered: hello
myshell> ls
You entered: ls
myshell> exit
Exiting shell...
akshitha_reddy@Akshitha-Reddy:~/SkillWeek3$ |
```

GNU nano 8.7.1

```
#include <stdio.h>
#include <string.h>

int main() {
    char input[100];

    while (1) {
        printf("myshell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL) {
            break;
        }

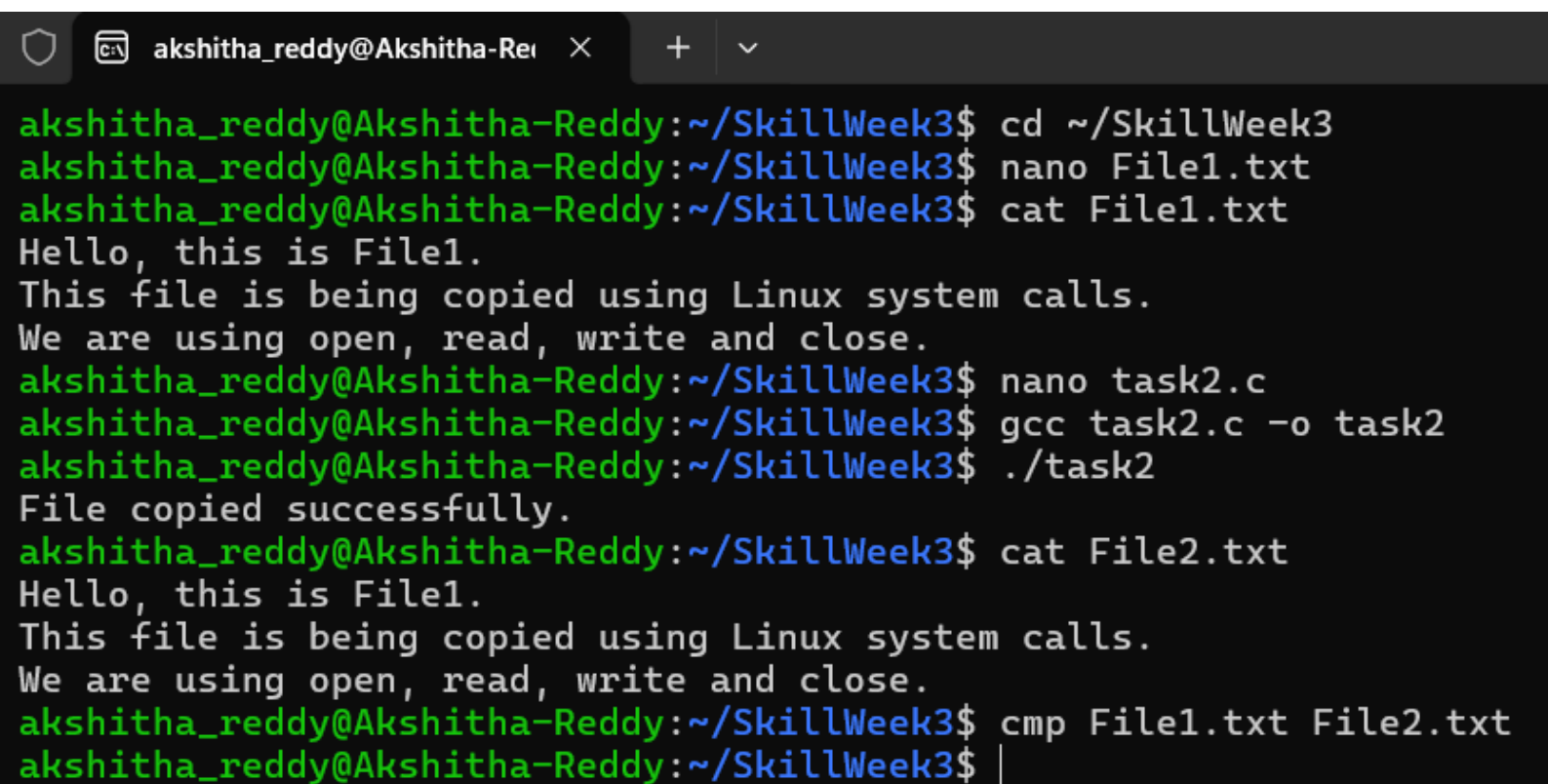
        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0) {
            printf("Exiting shell...\n");
            break;
        }

        if (strlen(input) == 0) {
            continue;
        }

        printf("You entered: %s\n", input);
    }

    return 0;
}
```



A terminal window with a dark background and light-colored text. The window title bar shows a shield icon, a tab labeled 'akshitha_reddy@Akshitha-Rei', and window control buttons. The terminal content shows a series of commands and their outputs in a monospaced font. The user is in the directory ~/SkillWeek3. They create a file File1.txt, view its contents, copy it using system calls, compile a C program task2.c, and compare the original and copied files.

```
akshitha_reddy@Akshitha-Reddy:~/SkillWeek3$ cd ~/SkillWeek3
akshitha_reddy@Akshitha-Reddy:~/SkillWeek3$ nano File1.txt
akshitha_reddy@Akshitha-Reddy:~/SkillWeek3$ cat File1.txt
Hello, this is File1.
This file is being copied using Linux system calls.
We are using open, read, write and close.
akshitha_reddy@Akshitha-Reddy:~/SkillWeek3$ nano task2.c
akshitha_reddy@Akshitha-Reddy:~/SkillWeek3$ gcc task2.c -o task2
akshitha_reddy@Akshitha-Reddy:~/SkillWeek3$ ./task2
File copied successfully.
akshitha_reddy@Akshitha-Reddy:~/SkillWeek3$ cat File2.txt
Hello, this is File1.
This file is being copied using Linux system calls.
We are using open, read, write and close.
akshitha_reddy@Akshitha-Reddy:~/SkillWeek3$ cmp File1.txt File2.txt
akshitha_reddy@Akshitha-Reddy:~/SkillWeek3$ |
```



akshitha_reddy@Akshitha-Rei



GNU nano 8.7.1

Hello, this is File1.

This file is being copied using Linux system calls.

We are using open, read, write and close.



GNU nano 8.7.1

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd1, fd2;
    char buffer[100];
    ssize_t bytesRead;

    fd1 = open("File1.txt", O_RDONLY);

    if (fd1 == -1) {
        perror("Error opening File1.txt");
        return 1;
    }

    fd2 = open("File2.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);

    if (fd2 == -1) {
        perror("Error opening File2.txt");
        close(fd1);
        return 1;
    }

    while ((bytesRead = read(fd1, buffer, sizeof(buffer))) > 0) {
        if (write(fd2, buffer, bytesRead) != bytesRead) {
            perror("Error writing to File2.txt");
            close(fd1);
            close(fd2);
            return 1;
        }
    }

    if (bytesRead == -1) {
        perror("Error reading File1.txt");
    }

    close(fd1);
    close(fd2);

    printf("File copied successfully.\n");

    return 0;
}
```

```
akshitha_reddy@Akshitha-Reddy:~/SkillWeek3$ nano task3.c
akshitha_reddy@Akshitha-Reddy:~/SkillWeek3$ gcc task3.c -o task3
akshitha_reddy@Akshitha-Reddy:~/SkillWeek3$ ./task3
Enter command: hello
You entered: hello
akshitha_reddy@Akshitha-Reddy:~/SkillWeek3$ ./task3
Enter command: hellox
You entered: hellox
akshitha_reddy@Akshitha-Reddy:~/SkillWeek3$ |
```

GNU nano 8.7.1

```
#include <stdio.h>
#include <unistd.h>
#include <termios.h>

#define BUFFER_SIZE 100

int main() {
    struct termios oldt, newt;
    char buffer[BUFFER_SIZE];
    int position = 0;
    char ch;

    tcgetattr(STDIN_FILENO, &oldt);
    newt = oldt;

    newt.c_lflag &= ~(ICANON | ECHO);

    tcsetattr(STDIN_FILENO, TCSANOW, &newt);

    printf("Enter command: ");
    fflush(stdout);

    while (1) {
        read(STDIN_FILENO, &ch, 1);

        if (ch == '\n' || ch == '\r') {
            buffer[position] = '\0';

            printf("\nYou entered: %s\n", buffer);
            break;
        }

        if (ch == 127 || ch == 8) {
            if (position > 0) {
                position--;

                printf("\b \b");
            }
        }
    }
}
```

```
position, &ch);
```

```
    printf("\b \b");  
    fflush(stdout);  
}
```

```
}  
else if (position < BUFFER_SIZE - 1) {  
    buffer[position++] = ch;
```

```
    write(STDOUT_FILENO, &ch, 1);  
    fflush(stdout);  
}
```

```
}
```

```
tcsetattr(STDIN_FILENO, TCSANOW, &oldt);
```

```
return 0;
```

```
}
```

```
akshitha_reddy@Akshitha-Reddy:~/SkillWeek3$ pwd
/home/akshitha_reddy/SkillWeek3
akshitha_reddy@Akshitha-Reddy:~/SkillWeek3$ ls
File1.txt  task1      task2      task2.c.save  task3.c
File2.txt  task1.c    task2.c    task3
akshitha_reddy@Akshitha-Reddy:~/SkillWeek3$ nano task4.c
akshitha_reddy@Akshitha-Reddy:~/SkillWeek3$ gcc task4.c -o task4
akshitha_reddy@Akshitha-Reddy:~/SkillWeek3$ ./task4
myshell> hello
Command: hello
myshell> linux
Command: linux
myshell> ubuntu
Command: ubuntu
myshell> Akshitha
Command: Akshitha
myshell> exit
akshitha_reddy@Akshitha-Reddy:~/SkillWeek3$ |
```



GNU nano 8.7.1

```
#include <stdio.h>
#include <unistd.h>
#include <termios.h>
#include <string.h>

#define BUFFER_SIZE 100
#define HISTORY_SIZE 10

void clear_line(int length) {
    for (int i = 0; i < length; i++) {
        printf("\b \b");
    }
    fflush(stdout);
}

int main() {
    struct termios oldt, newt;

    char buffer[BUFFER_SIZE];
    char history[HISTORY_SIZE][BUFFER_SIZE];

    int position;
    int history_count = 0;
    int history_index = 0;

    char ch;
    char escape[2];

    tcgetattr(STDIN_FILENO, &oldt);
    newt = oldt;

    newt.c_lflag &= ~(ICANON | ECHO);

    tcsetattr(STDIN_FILENO, TCSANOW, &newt);

    while (1) {
        position = 0;
        buffer[0] = '\0';

        printf("myshell> ");
        fflush(stdout);
```



```
while (1) {
    read(STDIN_FILENO, &ch, 1);

    if (ch == '\n' || ch == '\r') {
        buffer[position] = '\0';

        printf("\n");

        if (strcmp(buffer, "exit") == 0) {
            tcsetattr(STDIN_FILENO, TCSANOW, &oldt);
            return 0;
        }

        if (position > 0 && history_count < HISTORY_SIZE) {
            strcpy(history[history_count], buffer);
            history_count++;
        }

        printf("Command: %s\n", buffer);
        break;
    }

    if (ch == 127 || ch == 8) {
        if (position > 0) {
            position--;
            buffer[position] = '\0';

            printf("\b \b");
            fflush(stdout);
        }
    }

    else if (ch == 27) {
        read(STDIN_FILENO, &escape[0], 1);

        if (escape[0] == '[') {
            read(STDIN_FILENO, &escape[1], 1);

            if (escape[1] == 'A') {
                if (history_count > 0 && history_index > 0) {
```

```

        if (escape[1] == 'A') {
            if (history_count > 0 && history_index > 0) {
                clear_line(position);

                history_index--;

                strcpy(buffer, history[history_index]);
                position = strlen(buffer);

                printf("%s", buffer);
                fflush(stdout);
            }
        }

        else if (escape[1] == 'B') {
            if (history_index < history_count - 1) {
                clear_line(position);

                history_index++;

                strcpy(buffer, history[history_index]);
                position = strlen(buffer);

                printf("%s", buffer);
                fflush(stdout);
            }
        }
    }

    else if (position < BUFFER_SIZE - 1) {
        buffer[position++] = ch;
        buffer[position] = '\0';

        write(STDOUT_FILENO, &ch, 1);
        fflush(stdout);
    }
}

history_index = history_count;

```

```
        history_index = history_count;
    }

    tcsetattr(STDIN_FILENO, TCSANOW, &oldt);

    return 0;
}
```

```
akshitha_reddy@Akshitha-Rei X + v
akshitha_reddy@Akshitha-Reddy:~/SkillWeek3$ valgrind --version
Command 'valgrind' not found, but can be installed with:
sudo snap install valgrind # version 3.26.0, or
sudo apt install valgrind # version 1:3.26.0-0ubuntu1
See 'snap info valgrind' for additional versions.
akshitha_reddy@Akshitha-Reddy:~/SkillWeek3$ sudo apt update
[sudo: authenticate] Password:
Hit:1 http://archive.ubuntu.com/ubuntu resolute InRelease
Get:2 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Get:3 http://archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Get:4 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Packages [371 kB]
Hit:5 http://archive.ubuntu.com/ubuntu resolute-backports InRelease
Get:6 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Packages [446 kB]
Get:7 http://security.ubuntu.com/ubuntu resolute-security/main Translation-en [89.6 kB]
Get:8 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Components [32.8 kB]
Get:9 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Components [43.1 kB]
Get:10 http://security.ubuntu.com/ubuntu resolute-security/restricted amd64 Packages [321 kB]
Get:11 http://security.ubuntu.com/ubuntu resolute-security/restricted Translation-en [61.2 kB]
Get:12 http://archive.ubuntu.com/ubuntu resolute-updates/main Translation-en [111 kB]
Get:13 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Components [84.7 kB]
Get:14 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Packages [235 kB]
Get:15 http://archive.ubuntu.com/ubuntu resolute-updates/universe Translation-en [76.9 kB]
Get:16 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Components [166 kB]
Get:17 http://archive.ubuntu.com/ubuntu resolute-updates/restricted amd64 Packages [331 kB]
Get:18 http://archive.ubuntu.com/ubuntu resolute-updates/restricted Translation-en [63.0 kB]
Get:19 http://archive.ubuntu.com/ubuntu resolute-updates/multiverse amd64 Packages [11.4 kB]
Get:20 http://archive.ubuntu.com/ubuntu resolute-updates/multiverse Translation-en [3032 B]
Fetched 2723 kB in 9s (295 kB/s)
40 packages can be upgraded. Run 'apt list --upgradable' to see them.
akshitha_reddy@Akshitha-Reddy:~/SkillWeek3$ sudo apt install valgrind
Installing:
  valgrind

Installing dependencies:
  gdb          libdebuginfod-common libipt2
  libbabeltrace1 libdebuginfod1t64 libsource-highlight-common
  libc6-dbg     libdw1t64             libsource-highlight4t64

Suggested packages:
  gdb-doc gdbserver valgrind-mpi kcachegrind

Summary:
  Upgrading: 0, Installing: 10, Removing: 0, Not Upgrading: 40
  Download size: 27.1 MB
  Space needed: 111 MB / 1024 GB available
```

```

Processing triggers for libc-bin (2.43-2ubuntu2.3) ...
akshitha_reddy@Akshitha-Reddy:~/SkillWeek3$ nano task5.c
akshitha_reddy@Akshitha-Reddy:~/SkillWeek3$ gcc -g task5.c -o task5
akshitha_reddy@Akshitha-Reddy:~/SkillWeek3$ ./task5
Enter a command: This is my dynamic buffer test

Command stored: This is my dynamic buffer test
akshitha_reddy@Akshitha-Reddy:~/SkillWeek3$ valgrind --leak-check=full ./task5
==2039== Memcheck, a memory error detector
==2039== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==2039== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==2039== Command: ./task5
==2039==
Enter a command: Hello Ubuntu

Command stored: Hello Ubuntu
==2039==
==2039== HEAP SUMMARY:
==2039==      in use at exit: 0 bytes in 0 blocks
==2039==    total heap usage: 6 allocs, 6 frees, 2,107 bytes allocated
==2039==
==2039== All heap blocks were freed -- no leaks are possible
==2039==
==2039== For lists of detected and suppressed errors, rerun with: -s
==2039== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
akshitha_reddy@Akshitha-Reddy:~/SkillWeek3$ cd ~/SkillWeek3
akshitha_reddy@Akshitha-Reddy:~/SkillWeek3$ ls -l
total 124
-rw-r--r-- 1 akshitha_reddy akshitha_reddy  116 Aug  7 13:58 File1.txt
-rw-r--r-- 1 akshitha_reddy akshitha_reddy  116 Aug  7 13:59 File2.txt
-rwxr-xr-x 1 akshitha_reddy akshitha_reddy 16304 Aug  7 08:16 task1
-rw-r--r-- 1 akshitha_reddy akshitha_reddy   526 Aug  7 08:38 task1.c
-rwxr-xr-x 1 akshitha_reddy akshitha_reddy 16216 Aug  7 13:59 task2
-rw-r--r-- 1 akshitha_reddy akshitha_reddy   891 Aug  7 13:59 task2.c
-rw----- 1 akshitha_reddy akshitha_reddy   891 Aug  7 08:36 task2.c.save
-rwxr-xr-x 1 akshitha_reddy akshitha_reddy 16264 Aug  7 14:06 task3
-rw-r--r-- 1 akshitha_reddy akshitha_reddy  1019 Aug  7 14:08 task3.c
-rwxr-xr-x 1 akshitha_reddy akshitha_reddy 16472 Aug  7 14:10 task4
-rw-r--r-- 1 akshitha_reddy akshitha_reddy   3123 Aug  7 14:09 task4.c
-rw-r--r-- 1 akshitha_reddy akshitha_reddy   3123 Aug  7 14:14 'task4.c\'
-rwxr-xr-x 1 akshitha_reddy akshitha_reddy 18464 Aug  7 14:16 task5
-rw-r--r-- 1 akshitha_reddy akshitha_reddy   1699 Aug  7 14:16 task5.c
akshitha_reddy@Akshitha-Reddy:~/SkillWeek3$ |

```



GNU nano 8.7.1

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Node {
    char *command;
    struct Node *next;
} Node;

int main() {
    char *buffer;
    int size = 10;
    int length = 0;
    int ch;

    Node *head = NULL;
    Node *tail = NULL;

    buffer = malloc(size);

    if (buffer == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }

    printf("Enter a command: ");

    while ((ch = getchar()) != '\n' && ch != EOF) {

        if (length >= size - 1) {
            size *= 2;

            char *temp = realloc(buffer, size);

            if (temp == NULL) {
                free(buffer);
                printf("Memory reallocation failed.\n");
                return 1;
            }

            buffer = temp;
        }
    }
}
```



GNU nano 8.7.1

}

buffer[length++] = ch;

}

buffer[length] = '\0';

Node *newNode = malloc(sizeof(Node));

if (newNode == NULL) {

free(buffer);

printf("Node allocation failed.\n");

return 1;

}

newNode->command = malloc(strlen(buffer) + 1);

if (newNode->command == NULL) {

free(newNode);

free(buffer);

printf("Command allocation failed.\n");

return 1;

}

strcpy(newNode->command, buffer);

newNode->next = NULL;

if (head == NULL) {

head = newNode;

tail = newNode;

} else {

tail->next = newNode;

tail = newNode;

}

printf("\nCommand stored: %s\n", newNode->command);

free(buffer);

Node *current = head;

```
while (current != NULL) {  
    Node *next = current->next;  
  
    free(current->command);  
    free(current);  
  
    current = next;  
}  
  
return 0;  
}
```